Task 1b

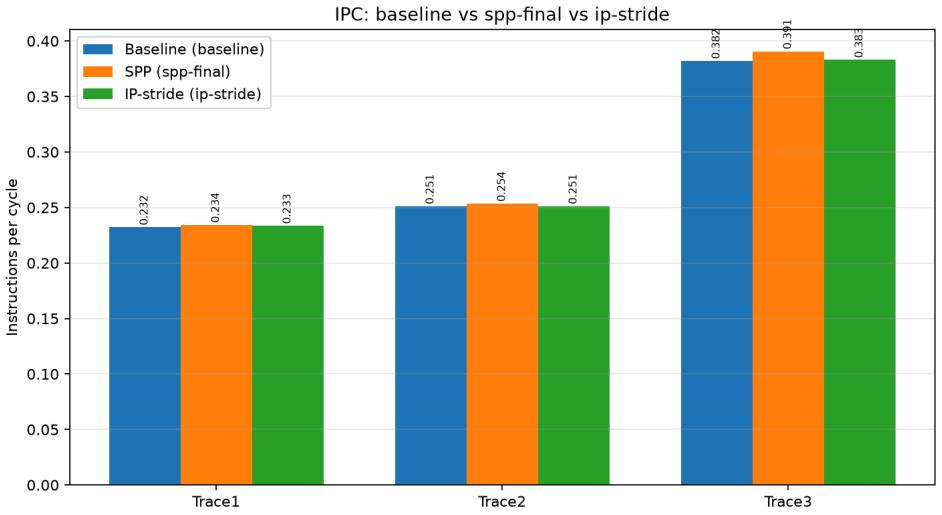
1. Results

The following results compare the baseline, the final SPP prefetcher, and the IP-stride prefetcher on the three traces.

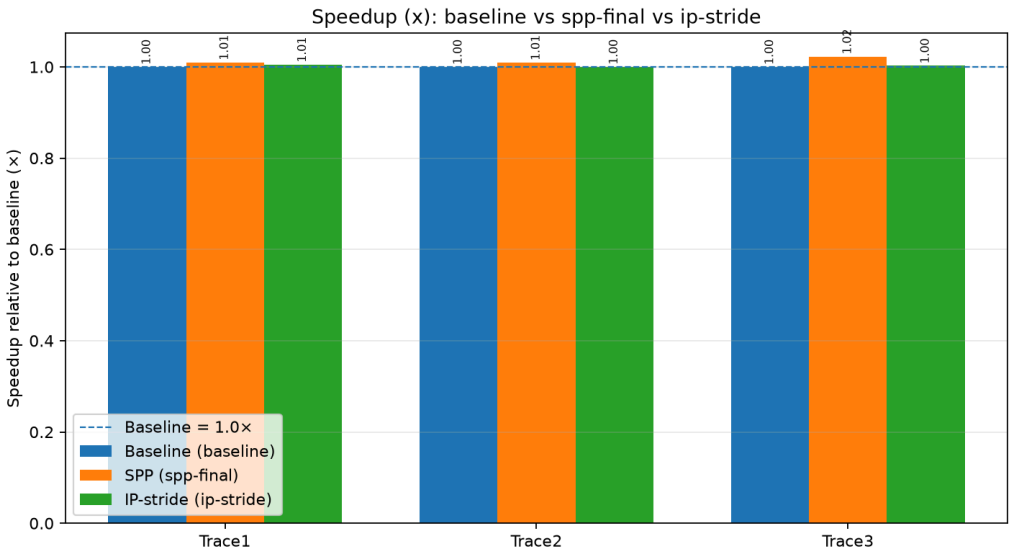
Trace	Prefetcher	IPC	Speedup	L1D Load MPKI	Accuracy (%)	Coverage (%)
Trace1	Baseline	0.232204	1.0000x	10.6787	N/A	0.000
Trace1	SPP	0.234491	1.0098x	9.5725	56.385	10.359
Trace1	IP-stride	0.233398	1.0051x	10.1188	52.791	5.243
Trace2	Baseline	0.251130	1.0000x	13.5338	N/A	0.000
Trace2	SPP	0.253504	1.0095x	13.1153	52.866	3.092
Trace2	IP-stride	0.251186	1.0002x	13.4356	38.128	0.725
Trace3	Baseline	0.382061	1.0000x	3.4032	N/A	0.000
Trace3	SPP	0.390760	1.0228x	2.8383	50.793	16.601
Trace3	IP-stride	0.382962	1.0024x	3.3177	16.094	2.514

The SPP prefetcher improves IPC on all three traces. Its largest speedup is 1.0228x on Trace3. It also gives lower L1D load MPKI than the baseline on all three traces.

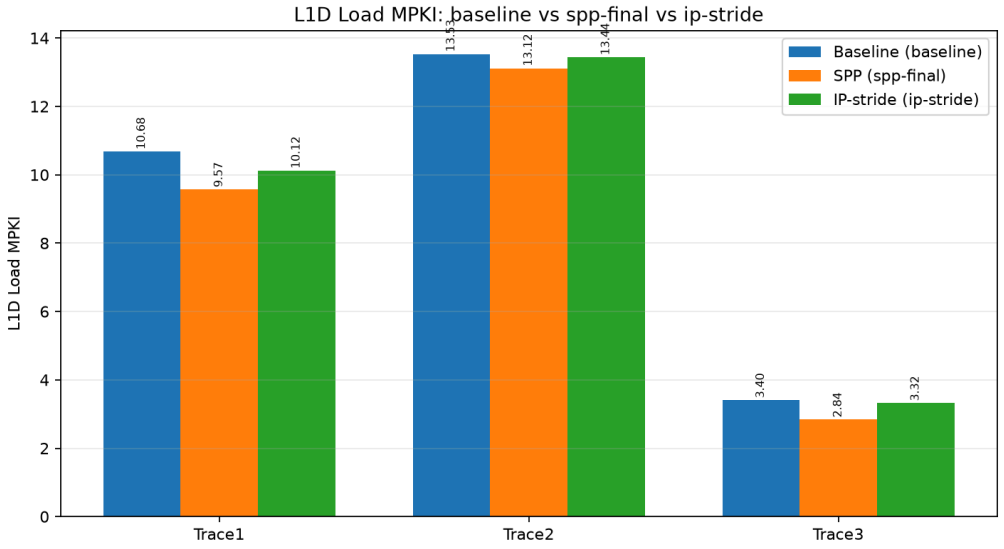
Plots



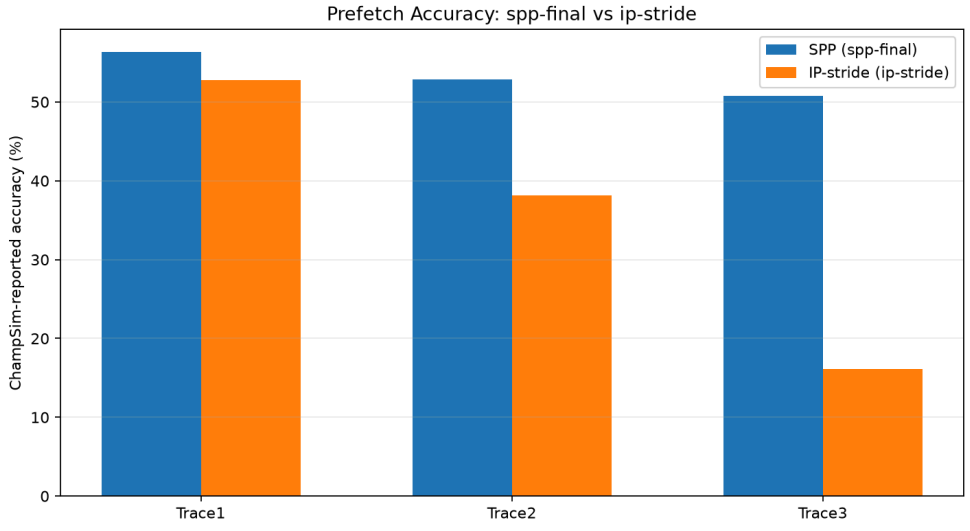
IPC



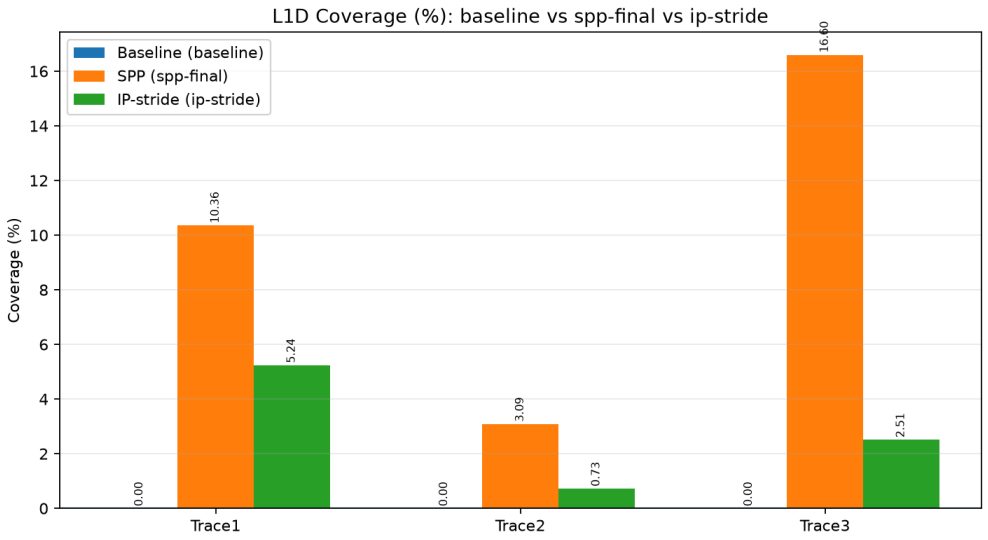
Speedup



L1D Load MPKI



Prefetch accuracy



L1D **coverage**

2. Storage overhead

The storage calculation below uses the bit widths needed by the new compressed code. It is a hardware storage calculation, so compiler padding in the C++ simulator is not included.

IP tracker table

Field	Bits	Purpose
Valid	1	Shows whether the entry is in use
IP tag	8	Checks whether the indexed entry belongs to the current IP
Last cache-line address	16	Only the low 16 bits are kept because the code only needs the recent small stride
Last stride	7	The accepted stride range is -64 to 63
Confidence	2	Values are 0 to 3
Signature	12	The signature is masked with 0xFFF
Use pattern	1	Selects uniform or pattern prediction

Each IP entry uses 47 bits. There are 128 entries, so the IP tracker table uses 47 x 128 = 6,016 bits. The 7-bit table index is not stored in the entry because the array position already provides it.

Pattern table

Field	Bits	Purpose
Signature tag	4	The 12-bit signature uses 8 bits for the table index, leaving 4 tag bits
Next stride	7	Stores a stride from -64 to 63
Confidence	2	Values are 0 to 3; 0 also represents an empty entry

Each pattern entry uses 13 bits. There are 256 entries, so the pattern table uses 13 x 256 = 3,328 bits.

Accuracy throttling state

State	Bits	Reason
Useful count for the epoch	9	Needs to represent values up to 256
Useless count for the epoch	9	Needs to represent values up to 256
Prefetch degree	4	Stores the literal values 1, 2, 4, or 8

The throttling state uses 22 bits. Therefore the total hardware storage is 6,016 + 3,328 + 22 = 9,366 bits. This is 1,170.75 bytes, or about 1.14 KiB. The request and miss counters printed only for the final report are simulator statistics, so they are not counted as prefetcher hardware storage.

3. Accuracy throttling threshold

The prefetcher checks its accuracy after 256 completed prefetches. For one epoch, good is the number of useful prefetches and bad is the number of useless prefetches. The code calculates accuracy as 100 x good / (good + bad).

I used thresholds of 25%, 50%, and 75%. These divide the accuracy into four simple ranges. I kept these thresholds after the parameter tests because changing them did not give a better overall result. The 256-prefetch epoch also stops the degree from changing because of only a few random prefetches.

Accuracy range	Prefetch degree
< 25%	1
25% to < 50%	2
50% to < 75%	4
>= 75%	8

4. Why these prefetch degrees were used

The degree is increased only when the prefetcher is more accurate. When accuracy is below 25%, degree 1 is used so that a poor predictor does not send many wrong requests. From 25% to 50%, degree 2 is a small increase. From 50% to 75%, degree 4 gives more coverage while still limiting cache and memory traffic. At 75% or higher, degree 8 is used because the predictor is reliable enough to look farther ahead.

The values 1, 2, 4, and 8 also make the change gradual because the degree doubles at each level. The maximum degree of 8 was kept because it gave a small performance improvement in the degree sweep. This gives an aggressive setting only when accuracy is high and a conservative setting when accuracy is low.

Task 2a - Instruction-Conscious L2 Replacement Policy (I-RRIP)

2.1 Results

Table 2a-1: IPC and speedup over SRRIP

Trace	IPC (RRIP)	IPC (I-RRIP)	Speedup
Trace 1	0.232007	0.231939	0.9997
Trace 2	0.251127	0.255560	1.0177
Trace 3	0.381616	0.381982	1.0010

Table 2a-2: L2C load MPKI (total, instruction, data)

Trace	Policy	L2C Load MPKI	L2 Instr. Load MPKI	L2 Data Load MPKI
Trace 1	RRIP	17.5872	10.5207	7.0664
Trace 1	I-RRIP	17.7327	9.8447	7.8880
Trace 2	RRIP	26.8715	18.9032	7.9683
Trace 2	I-RRIP	24.0119	14.6497	9.3622
Trace 3	RRIP	4.0470	1.2016	2.8454
Trace 3	I-RRIP	3.8935	1.0021	2.8914

Table 2a-3: Change of I-RRIP relative to RRIP

Trace	Instr. load MPKI	Data load MPKI	Total load MPKI
Trace 1	-6.4%	+11.6%	+0.8%
Trace 2	-22.5%	+17.5%	-10.6%
Trace 3	-16.6%	+1.6%	-3.8%

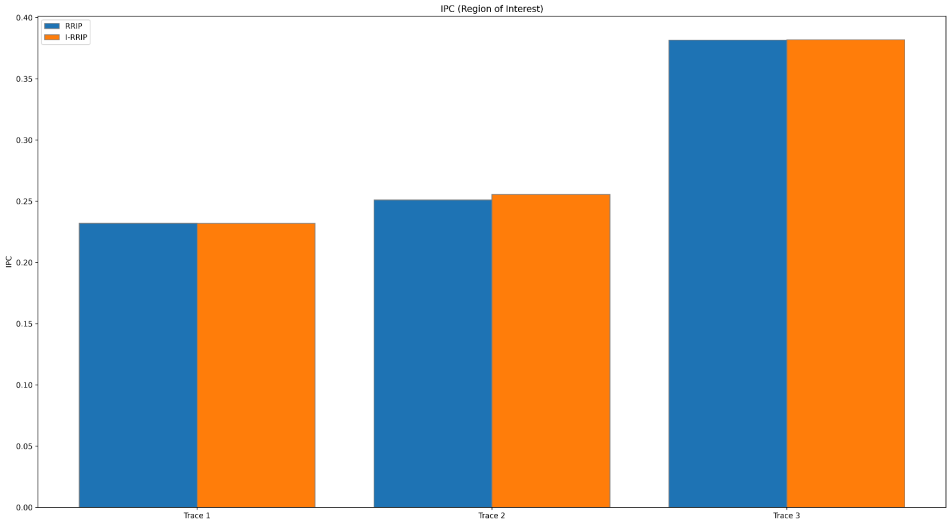


Figure 2a-1: Speedup of I-RRIP over RRIP per trace

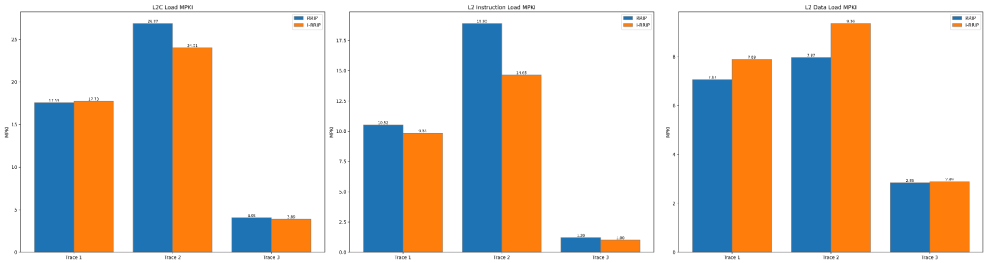


Figure 2a-2: L2C total / instruction / data load MPKI, RRIP vs I-RRIP

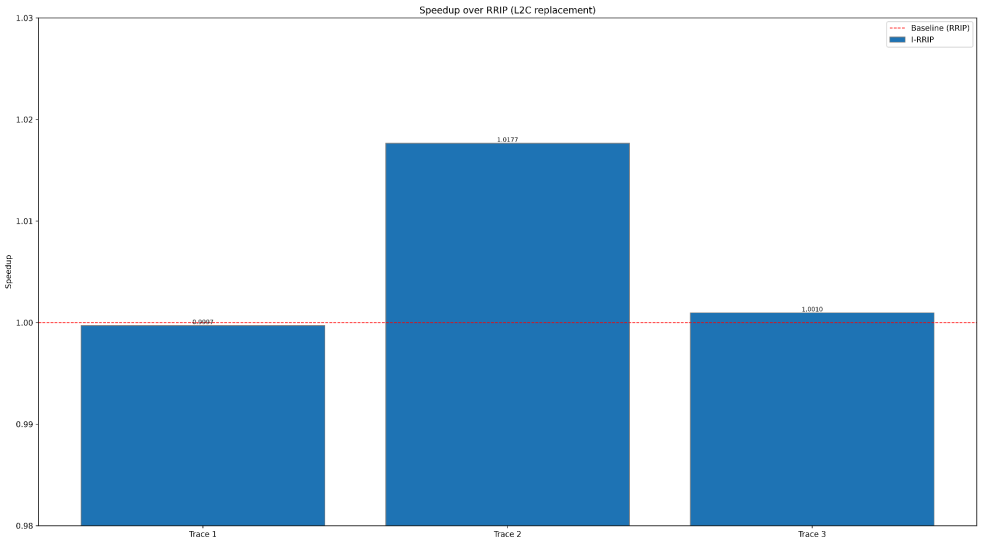


Figure 3: Speedup over RRIP

2.2 Analysis

Instruction misses go down on every trace. The reduction scales with how instruction-bound the trace is: Trace 2 (L1I MPKI 40.0) improves by 22.5% and is the only

trace with a clear IPC gain (+1.8%); its L1I average miss latency also falls from 40.6 to 37.6 cycles because more instruction misses are served by L2. Trace 3 is data-dominated (L1I MPKI 2.1), so the instruction gain is small in absolute terms and IPC is essentially flat (+0.1%).

Trace 1 shows no IPC benefit. Instruction MPKI falls only 6.4%, while data load MPKI rises 11.6%, so total load MPKI is slightly worse. The 0.9997 speedup is within noise and should be read as "no change".

Instruction protection has a cost. Prioritising instruction lines raises data load MPKI on Traces 1 and 2. On Trace 1 the total L2 miss count also rises by about 30K (of which writeback misses account for about 16K and translation loads for about 5K), and on Trace 2 writeback misses go from about 16K to about 51K. Our working hypotheses, to be confirmed by the variant experiments in Section 2.5, are that (i) decrementing instead of resetting RRPV on data hits reduces the retention of reused data and writeback lines, and (ii) page-walk (translation) requests are classified as data and are therefore treated as low priority.

2.3 Rationale for modifying insertion, promotion and eviction

Insertion. In server-style workloads the code footprint exceeds L1I, so an instruction line that was just fetched is likely to be fetched again soon and its miss stalls the whole frontend. Inserting instruction lines at RRPV 0 gives them the maximum initial protection, while data lines keep SRRIP's "long re-reference" insertion (RRPV 2), which retains scan resistance for streaming data. The difference between the two insertion values is what creates the instruction bias.

Promotion. An instruction hit resets RRPV to 0 (hit-priority), since a re-used instruction line is very likely to be re-used again. A data hit only moves the line one step closer to RRPV 0 (frequency-priority). This means a single touch cannot lift a data line to the same priority as hot code, and data must demonstrate repeated reuse to earn the same protection.

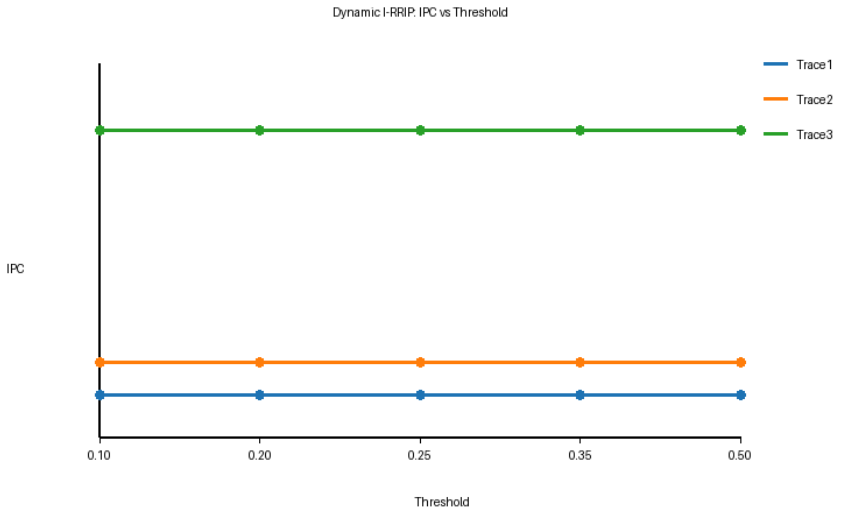
Eviction. Among all lines at RRPV 3 we evict a data line before an instruction line. This is a tie-break only: it does not override RRPV order, so a stale instruction line (one that has aged to RRPV 3 without being reused) can still be evicted when no data candidate exists, and aging applies to instruction and data lines alike. This prevents instruction lines from permanently polluting a set, which would be the failure mode of a hard "never evict instructions" rule, especially on traces where the instruction working set does not fit (instruction-on-instruction evictions are still the largest class of instruction-line evictions on Trace 2).

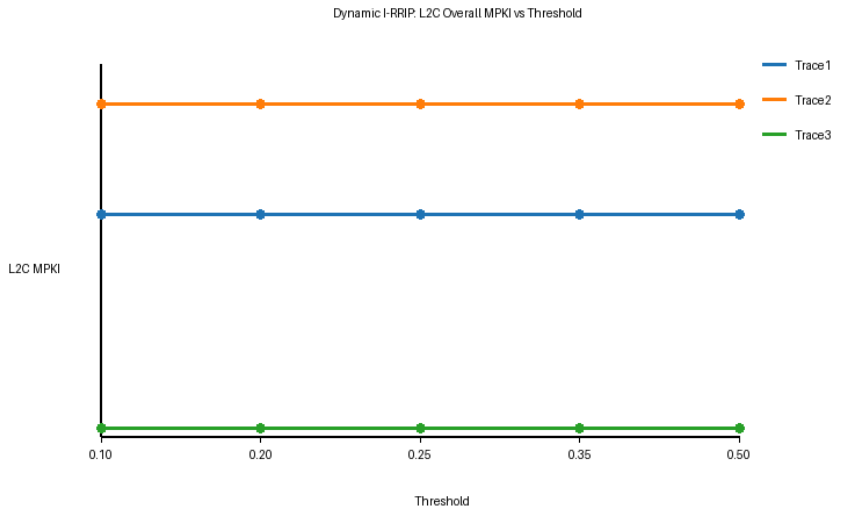
Why this design. Each change is small and independent, so its effect can be attributed, the added hardware is a single bit per line, and the aging rule bounds how long any line can be protected. Results above show it achieves the stated goal (lower L2 instruction load MPKI on all three traces) and improves IPC where instruction misses dominate; the side effects on data and translation traffic are the limits of a static policy and are what Task 2b's dynamic set-dueling approach is intended to address.

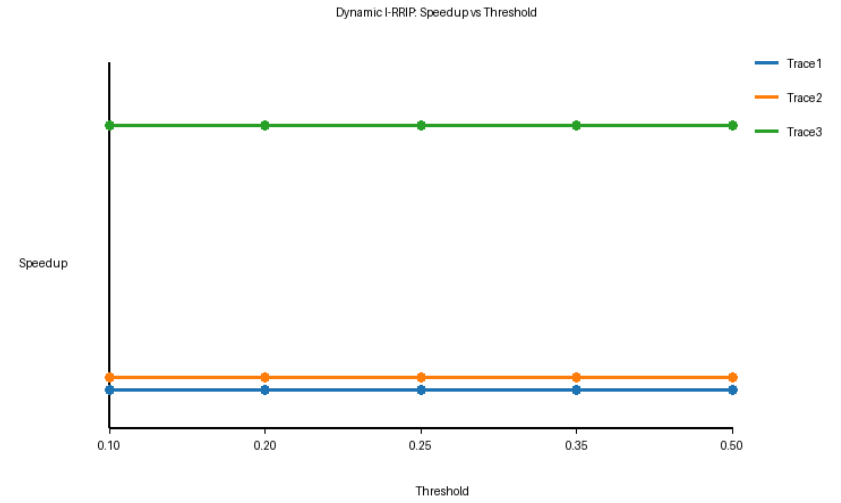
IPC, speedup, and Load MPKI of L2C, L2 Instruction, and L2 Data:

Threshold	Workload Trace	IPC	Speedup (vs. LRU Baseline)	L2C Total MPKI	L2C Instruction Load MPKI	L2C Data Load MPKI
0.10	Trace1	0.2321	0.6920	24.1563	10.4171	7.1699
0.10	Trace2	0.2509	0.7169	33.4574	18.8800	8.2113
0.10	Trace3	0.3817	1.1928	6.2944	1.1479	2.8280
0.20	Trace1	0.2321	0.6920	24.1563	10.4171	7.1699
0.20	Trace2	0.2509	0.7169	33.4574	18.8800	8.2113
0.20	Trace3	0.3817	1.1928	6.2944	1.1479	2.8280
0.25	Trace1	0.2321	0.6920	24.1563	10.4171	7.1699
0.25	Trace2	0.2509	0.7169	33.4574	18.8800	8.2113
0.25	Trace3	0.3817	1.1928	6.2944	1.1479	2.8280
0.35	Trace1	0.2321	0.6920	24.1563	10.4171	7.1699
0.35	Trace2	0.2509	0.7169	33.4574	18.8800	8.2113
0.35	Trace3	0.3817	1.1928	6.2944	1.1479	2.8280
0.50	Trace1	0.2321	0.6920	24.1563	10.4171	7.1699
0.50	Trace2	0.2509	0.7169	33.4574	18.8800	8.2113
0.50	Trace3	0.3817	1.1928	6.2944	1.1479	2.8280

Plots:







1. Leader Set Allocation

- Sampling Overhead: Dedicated 32 sets to Policy A (Instruction-Aware SRRIP) and 32 sets to Policy B (Instruction-Aware BRRIP). Sampling 64 total sets out of 1024 (approx 6.25%) minimizes hardware area while providing >95% statistical accuracy in workload profiling.
- Distribution: Sets are indexed using a bit-reversal hash to prevent spatial access locality from biasing policy selection. The remaining sets act as follower sets.

2. Decision Logic (PSEL)
   - Counter Mechanics: Uses a 10-bit saturating counter PSEL = [0, 1023], reset at 512. A miss on a Policy A set increments PSEL; a miss on a Policy B set decrements PSEL.
   - Threshold Evaluation: Follower sets execute Policy A if PSEL >= Floor(Theta x 1023), and Policy B otherwise. Instructions receive higher initial retention (RRPV = 0/1)compared to data (RRPV = M - 2) to protect critical code loops from data thrashing.

3. Flat Sensitivity Curves: Across all 3 traces, miss rates on one policy's sampling sets continuously dominate, keeping PSEL locked near boundary saturation. Consequently, varying Theta between 0.10 and 0.50 produces identical follower set behavior and flat performance lines.

Task 3: SPP with Prefetch-aware Dynamic I-RRIP

Results

Course traces, Task 3 configuration (SPP + prefetch-aware Dynamic I-RRIP, 25M warmup + 25M simulated). Trace 3 runs fastest (IPC 0.384) and has the lowest miss rates at every level. Trace 2 is the most front-end bound (L1I load MPKI 40.0, L2 instruction load MPKI 14.4). These runs used the build from before the prefetch duel (rule 6): the output has no `PF-aware duel` line, so rules 1-4 applied in every follower set.

Trace	IPC	L1D PF accuracy (%)	L1D load MPKI	L2C load MPKI	L2 instr. load MPKI	L2 data load MPKI	L1I load MPKI
Trace 1	0.2337	54.4	9.89	17.01	10.00	7.00	14.48
Trace 2	0.2560	42.7	13.37	23.32	14.43	8.89	40.02
Trace 3	0.3840	29.5	3.23	3.58	1.05	2.53	2.13

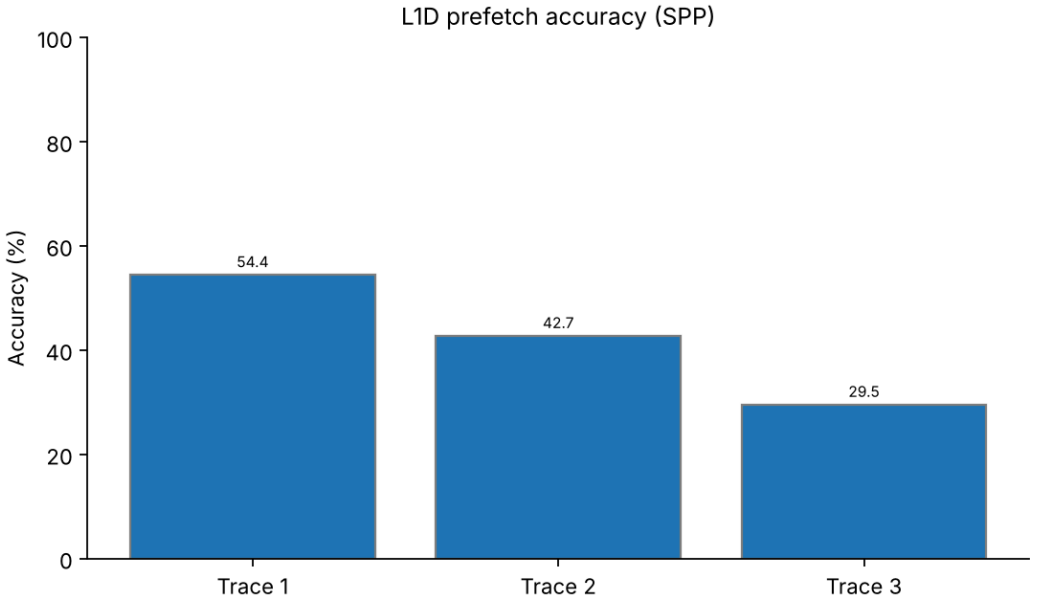
Speedup and coverage need the reference runs. Both are measured against the no-prefetch LRU baseline, which was not among the submitted outputs. Run `baseline`, `spp`, `dynirrip` and `spp_dynirrip_nopf` on the same traces and complete this table. Speedup = Task 3 IPC / baseline IPC. Coverage = (baseline L1D load misses - Task 3 L1D load misses) / baseline L1D load misses.**How the policy treated SPP's lines (from its own end-of-run counters).** Nearly every prefetched copy that SPP placed in L2 died there unused, which is the case rules 1 and 4 target.

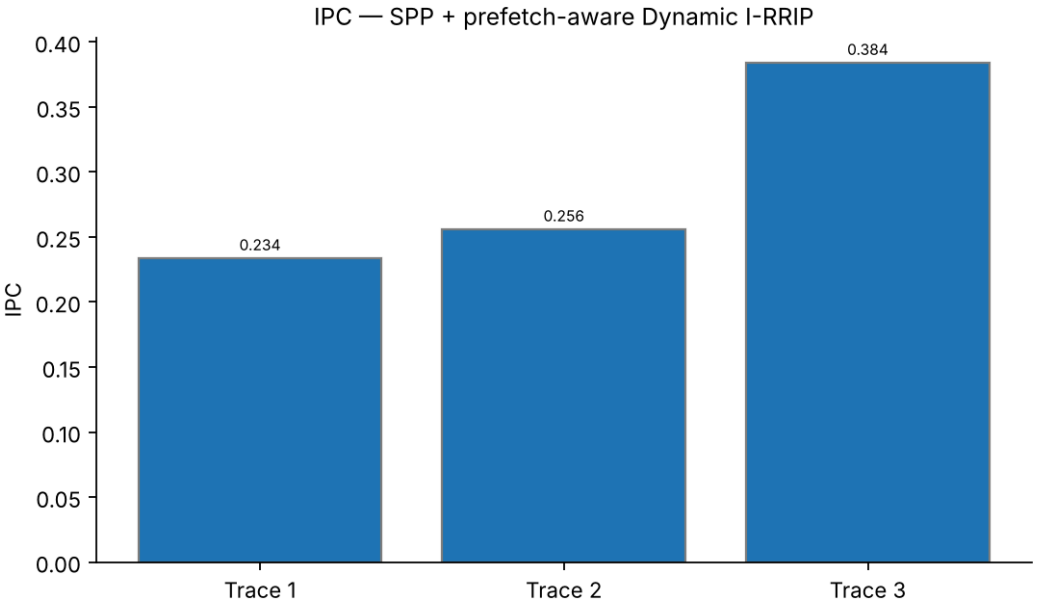
Trace	Follower accesses in I-RRIP mode (%)	PF fills inserted at RRPV 3 (%)	PF fills evicted unused (%)	L2 prefetch usefulness (%)	ChampSim L2C prefetch accuracy (%)
Trace 1	96.1	99.5	97.0	2.87	3.05
Trace 2	94.6	99.1	98.2	1.77	2.06
Trace 3	90.1	98.9	98.4	1.39	3.21

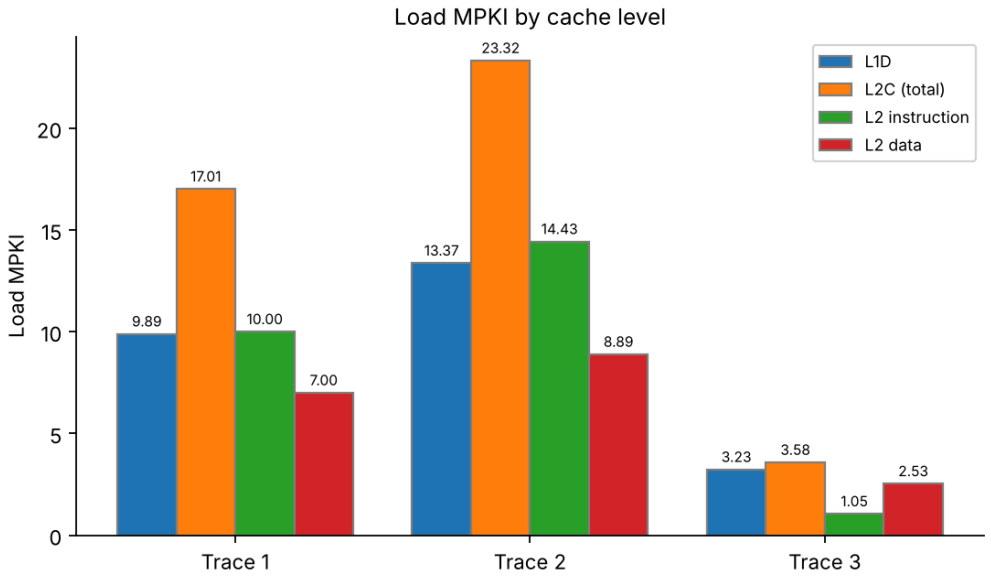
L2 prefetch usefulness = demand hits on prefetched L2 lines / (those hits + prefetched lines evicted unused). The usefulness counter ended at 0 on all three traces, so almost every prefetch was inserted at RRPV 3.

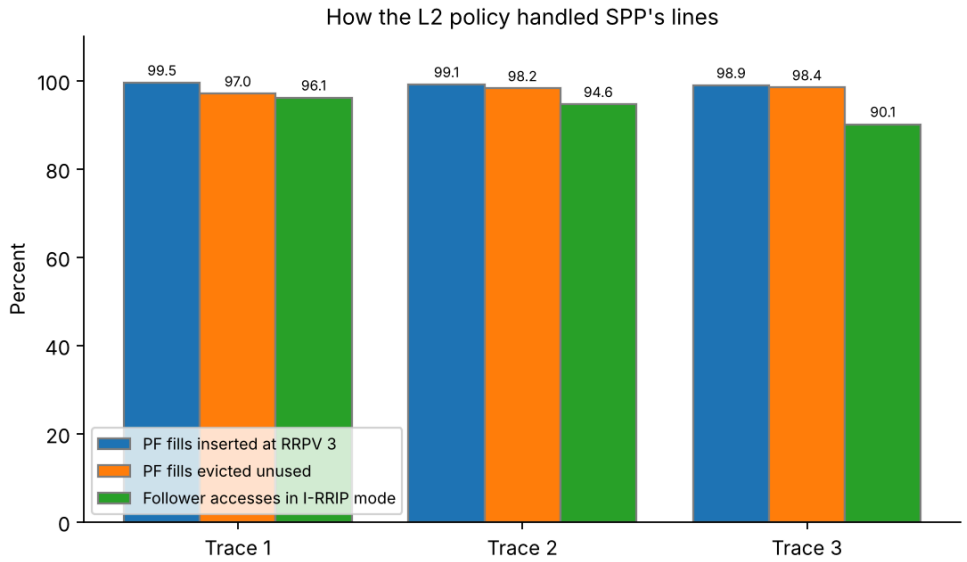
SPP behaviour. Throttling worked as designed: the lower a trace's accuracy, the lower its average degree. The signature (non-uniform) path issued most prefetches.

Trace	L1D accuracy (%)	Average degree	Accesses at degree 1 (%)	Uniform-stride prefetches	Signature prefetches	Timely prefetches	Late prefetches
Trace 1	54.4	2.43	24.5	174,103	529,523	23,995	4,167
Trace 2	42.7	1.73	68.1	105,571	259,874	6,254	1,578
Trace 3	29.5	1.56	63.8	174,697	478,085	4,711	4,302


L1D prefetch accuracy (SPP)
Accuracy (%)
100
80
60
40
20
0
54.4
42.7
29.5
Trace 1
Trace 2
Trace 3


IPC — SPP + prefetch-aware Dynamic I-RRIP
IPC
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0.234
0.256
0.384
Trace 1
Trace 2
Trace 3





Rationale for handling prefetch packets

The central observation: an L2 copy of an SPP prefetch is usually dead on arrival. Most SPP prefetches are issued with fill level L1, so the line is used from L1D and the L2 copy is never touched again. Under plain SRRIP that copy is inserted at RRPV 2, the same as a demand-fetched line. It therefore pushes out instruction lines and reused data lines that do have L2 reuse. Each rule below protects those lines from prefetch pollution without throwing away prefetches that are useful at L2.

1. **Distant insertion for prefetches, adjusted by measured usefulness.** Inserting at RRPV 3 makes a prefetched copy the first candidate for eviction, which is the right default for L1-fill prefetches. Some prefetches *are* consumed at L2: those issued with fill level L2 when the L1D MSHR is busy, and lines L1D evicts before use. The usefulness counter catches that case and moves insertion back to RRPV 2, so timely L2 prefetches are not evicted before their demand arrives. The counter measures

usefulness at L2 specifically, which is the quantity this decision needs, rather than L1D accuracy.

2. **No promotion on prefetch hits.** When an L1D prefetch hits in L2, L1D is pulling the line up and will serve the coming demand itself. The hit says nothing about future L2 reuse. Promoting it, as plain RRIP does, would protect a line that now lives in L1D anyway; this is the same reasoning as PACMan's prefetch-hit rule.
3. **Prefetched lines must prove reuse twice.** The first demand hit on a prefetched line is often its only use (streaming data). Promoting it to RRPV 1 instead of 0 keeps it ahead of fresh insertions but behind lines with real reuse; a second hit gives full protection. Instruction lines go straight to RRPV 0 so that I-RRIP's instruction priority is never weakened by prefetching.
4. **Unused prefetched lines are evicted first.** Among lines at RRPV 3, a line that was prefetched and never demanded has shown no value at all, so it is cheaper to lose than a demand-fetched data line or an instruction line. This is the rule that most directly protects instruction lines, reducing L2 instruction MPKI.
5. **Set dueling counts demand traffic only.** The Task 2B ratio (instr. acc + instr. miss) / (total acc + total miss) would count every SPP fill and prefetch hit as data. An aggressive prefetcher would push the ratio below 0.25 and switch the followers to SRRIP, removing instruction protection exactly when SPP is active. Counting only LOAD, RFO and LOAD_TRANSLATION keeps the decision tied to the program's real instruction/data mix.
6. **The prefetch rules are themselves set-duelled.** Rules 1-4 are heuristics; on some workloads, for example when most prefetches are fill-L2 and timely, plain handling may be better. A DIP-style duel lets the cache measure that directly: leader groups with and without the rules compete on demand misses. Followers adopt whichever group misses less, so the policy should not fall measurably below SPP + plain Dynamic I-RRIP. Instruction misses are weighted 2× because they stall the front end, while many data misses overlap with other work in the out-of-order core.

Taken together, SPP keeps its L1D coverage, since the L1D side is untouched. Meanwhile the L2 stops giving up instruction and reused data lines to prefetched copies. That is how the combination can beat both standalone SPP (which pollutes L2) and standalone Dynamic I-RRIP (which has no prefetch coverage).